

CS & IT ENGINEERING



Data Structure & Programming

Arrays

Lecture No.- 02



By- Satya sir

Recap of Previous Lecture



- Data Structure?
- Classification
 - Linear : Arrays, LL, Stack, Queue, Hash Table
 - Non-linear : Trees, Graphs, Sets, maps
- Efficiency of algorithm / Program : Time and Space Complexity
- Best, Average, Worst case
- $\Omega()$, $\Theta()$, $\underline{O()}$

Topics to be Covered



Array DS

- Definition, Nature
- Memory allocation
- Types of arrays - 1-D, 2-D, 3-D
- Access operations
- Insert "
- Delete "
- Search operations





Topic : 1-D Arrays



Array :

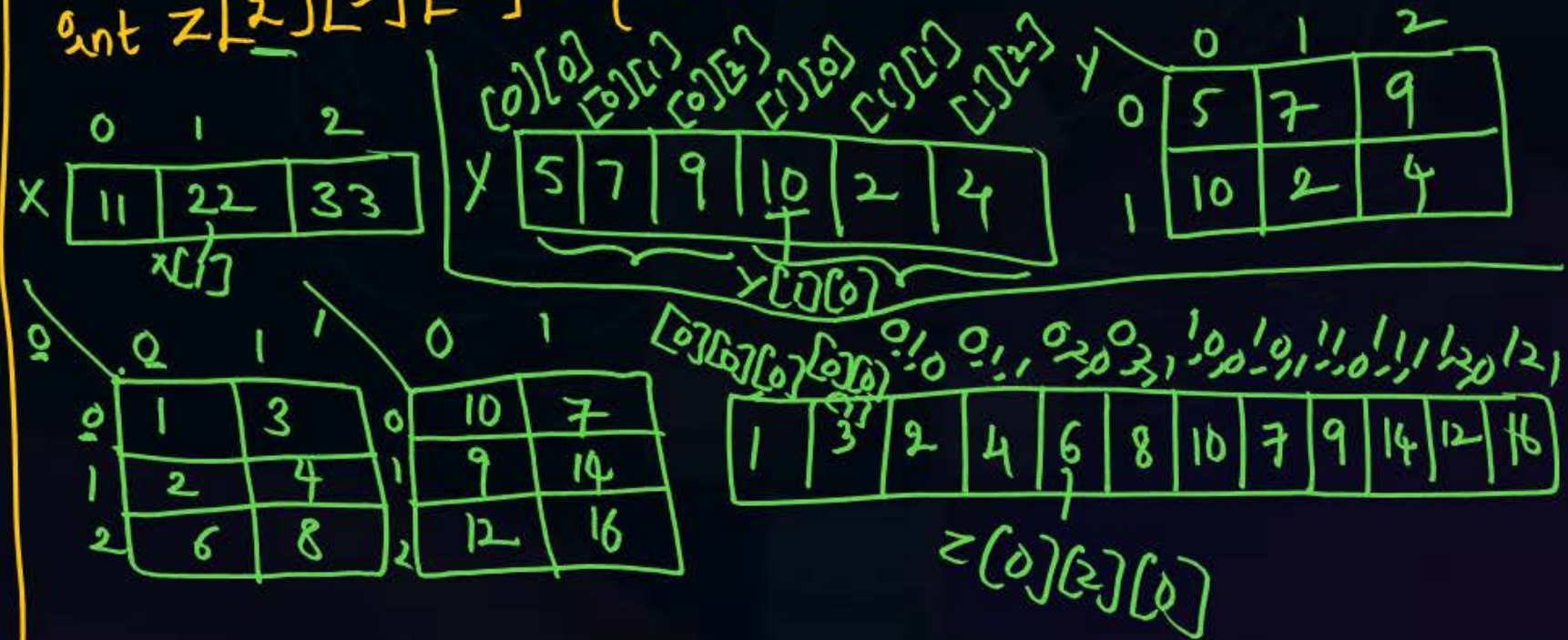
- Linear Data Structure
- Collection of Similar (or) Homogeneous data
- Reference / Base address of group of elements
- Constant Pointer
- Elements of array can be modified.
- Types of arrays :
 - 1) one-dimensional - 1-D
 - 2) Multi-dimensional - 2-D, 3-D

- Static / compile-time Memory allocation
- Contiguous memory allocation
- Random access is possible using index

Ex: $\text{int } x[3] = \{11, 22, 33\};$

$\text{int } y[2][3] = \{5, 7, 9, 10, 2, 4\};$

$\text{int } z[2][3][2] = \{1, 3, 2, 4, 6, 8, 10, 7, 9, 14, 12, 16\};$





Topic : 1-D Arrays

Ex:

Let 1 int = 4 bytes

int $x[5] = \{10, 20, 30, 40, 50\}$

	0	1	2	3	4
x	10	20	30	40	50
→	1000	1004	1008	1012	1016

printf("%d", x); // 1000

Array name indicates reference (or)
Base address of group of elements.



Random indexing

- Default indexing starts from 0. [C, C++, Java, Python follow it. Python do support Negative indexing]
- But Languages like FORTRAN, LISP Supports random indexing ($\pm$ ve)

Ex:1 int $x[5] = \{10, 20, 30, 40, 50\}$, start index = -2

	-2	-1	0	1	2
x	10	20	30	40	50

Ex:2 int $y[2][3] = \{1, 3, 5, 7, 9, 11\}$

row index starts with -2
col index starts with -4

y \	-4	-3	-2
-2	1	3	5
-1	7	9	11



Topic : 1-D Arrays



Address of an Element

1-D array $A[]$, B = Base address
 n = Size of Each Element
 i = index, x = Start index

$$A[i] = B + [(i - x) * n]$$

Ex:

$$RMO: 1000 + [(-1 - (-1)) * 4 + 0 - (-3)] * 4$$

$$= 1000 + 12 = 1012$$

$$CMO: 1000 + [0 - (-3)] * 3 + (-1 - (-1)) * 4$$

$$= 1000 + 36 = 1036$$

-2	-1	0	1	2
1	3	5	7	9
1000	1004	1008	1012	1016

$$x[0] = 1000 + [0 - (-2)] * 4$$

$$= 1000 + 8 = 1008$$

Ex: $\text{int } y[3][4] = \{1, 3, 5, 7, 9, 2, 4, 6, 8, 10, 14, 16\}$

2-D array : Can be stored in either RMO (or) CMO

$$RMO: A[i][j] = B + [(i - x) * C + (j - y)] * n$$

$$CMO: A[i][j] = B + [(j - y) * R + (i - x)] * n$$

	-3	-2	-1	0	
1	1	3	5	7	
0	9	2	4	6	
1	8	10	14	16	

	-3	-2	-1	0	
1	1	3	5	7	9
0	9	2	4	6	8
1	8	10	14	16	10

B = Base address

x = Row start index

R = Total Number of rows

i = Row index

n = Size of Each Element

y = Col start index

C = Total Number of cols

j = Col index



Topic : 2-D Arrays



Given an array of 'n' Elements. Search | Find for an Element (Key) in an array.

2 Ways of Searching Techniques

a) Linear Search : Compare with Each Element one by one

b) Binary Search : 1) Calculate mid, $mid = \frac{\text{Start index} + \text{Stop index}}{2}$

↓
[input must be in sorted order]
[Let ascending order]

2) Compare with mid Element.

if $key == array[mid]$, Search Successful

else if $key < array[mid]$, repeat Process for left half Sublist.

else repeat Process for right half Sublist.

3) Do Procedure recursively until Search Successful (or) unsuccessful.



Topic : 2-D Arrays



Ex:

int x[10] = {1, 3, 5, 7, 9, 12, 14, 16, 18, 20}

Let key = 6

	low	0	1	2	high	3	4	5	6	7	8	high	9
x		1	3	5	7	9	12	14	16	18	20		

Linear search:

```
for (i = 0; i < n; i++)
{
    if (key == x[i])
    {
        Pf("Key is found");
        break;
    }
}
```

Time complexity: $O(n)$

Binary Search

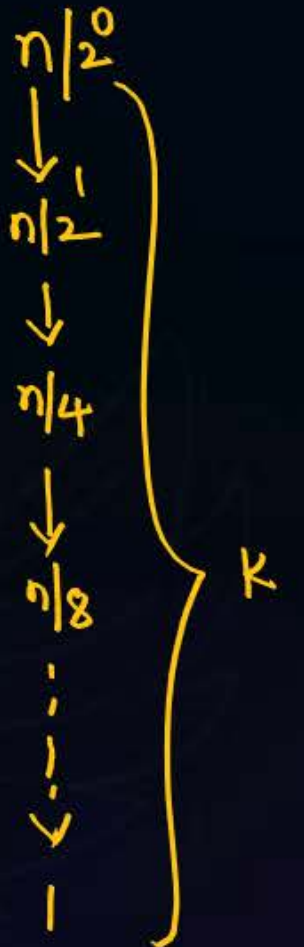
$$\text{mid} = \frac{0+9}{2} = \frac{9}{2} = 4$$

$$\text{key} == x[4] \Rightarrow \text{False}$$

$$\text{key} < x[4] \text{ True } \quad \text{high} = \text{mid} - 1$$

BSearch(low, high)

Time complexity: $O(\log_2^n)$



$$2^k = n$$

$$\Rightarrow k = \log_2^n$$

$$\Rightarrow \frac{n}{2^k} = 1$$



A one dimensional array A has indices 1....75. Each element is a string and takes up three memory words. The array is stored at location 1120 decimal. The starting address of A[49] is ____



Let A be an array containing integer values. The distance of A is defined as the minimum number of elements in A that must be replaced with another integer so that the resulting array is sorted in a non-decreasing order. The distance of the array [2, 5, 3, 1, 4, 2, 6] is _____



Suppose there are 11 items in sorted order in an array. How many searches are required on the average, if binary search is employed and all searches are successful in finding the item?

Ex:

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$
 0 1 2 3 4 5 6 7 8 9 10
10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110

if Key = 60 1 Search

if Key = 30 Compare (Key, 60), Compare (Key, 30) = 2

if Key = 90 Compare (Key, 60), Compare (Key, 70) = 2

if Key = 10 | 40 | 70 | 100
 $\left. \begin{array}{lll} (10, 60) & (40, 60) & (70, 60) \\ (10, 30) & (40, 30) & (70, 30) \\ (10, 10) & (40, 40) & (70, 70) \end{array} \right\} = 3$

if Key = 20 | 80 | 110 | 50

(20, 60), (20, 30), (20, 10), (20, 20)
 = 4 comparisons.

$$\frac{(1 \times 1) + (2 \times 2) + (4 \times 3) + (4 \times 4)}{11}$$

$$= \frac{1 + 4 + 12 + 16}{11} = \frac{33}{11} = \underline{\underline{3}}$$



Let A be an array of 31 numbers consisting of sequence of 0's followed by a sequence of 1's. The problem is to find the smallest index i that $A[i]$ is 1 by probing the minimum numbers of locations in A. The worst-case number of probes performed by an optimal algorithm is _____.



Let $A(1:8, -5:5, -10:5)$ be a three dimensional array. How many elements are there in the array A?

Count: last index – first index + 1

$$8 - 1 + 1 = 8$$

$$5 - (-5) + 1 = 11$$

$$5 - (-10) + 1 = 16$$

$$\begin{aligned} A[8][11][16] &\Rightarrow \text{Total elements} = 8 \times 11 \times 16 \\ &= 88 \times 16 \\ &= \underline{1408} \end{aligned}$$



An array A consists of n integers in locations $A[0], A[1] \dots A[n-1]$. It is required to shift the elements of the array cyclically to the left by k places, where $1 \leq k \leq (n-1)$. An incomplete algorithm for doing this in linear time, without using another array is given below. Complete the algorithm by filling in the blanks. Assume all the variables are suitably declared.

```
min = n; i = 0;
while (_____) {
    temp = A[i]; j = i;
    while (_____) {
        A[j] = _____
        j = (j + k) mod n ;
        If ( j < min ) then
            min = j;
    }
    A[(n + i - k) mod n] = _____
    i = _____
```

- A. $i > \text{min}; j \neq (n+i) \bmod n; A[j + k]; \text{temp}; i + 1;$
- B. $i < \text{min}; j \neq (n+i) \bmod n; A[j + k]; \text{temp}; i + 1;$
- C. $i > \text{min}; j \neq (n+i+k) \bmod n; A[(j + k)]; \text{temp}; i + 1;$
- D. $i < \text{min}; j \neq (n+i-k) \bmod n; A[(j + k) \bmod n]; \text{temp}; i + 1;$



Topic : 1-D Arrays



```
min = n; i = 0;
while ( _____ ) {
    temp = A[i]; j = i;
    while ( _____ ) {
        A[j] = _____
        j = (j + k) mod n ;
        If ( j < min ) then
            min = j;
    }
    A[(n + i - k) mod n] = _____
    i = _____
```

A. $i > \text{min}; j \neq (n+i) \bmod n; A[j + k]; \text{temp}; i + 1;$

B. $i < \text{min}; j \neq (n+i) \bmod n; A[j + k]; \text{temp}; i + 1;$

C. $i > \text{min}; j \neq (n+i+k) \bmod n; A[(j + k)]; \text{temp}; i + 1;$

D. $i < \text{min}; j \neq (n+i-k) \bmod n; A[(j + k) \bmod n]; \text{temp}; i + 1;$



2 mins Summary



Arrays

- Types
- Address of an element
- Access
- Search
 - linear Search
 - Binary Search

To be Contd... 



THANK - YOU